OPERATING SYSTEMS (UE24CS242B)

Unit 1 – CPU Scheduling

CPU Scheduling – Basic Concepts

In a computer system with a **single CPU core**, only **one process can execute at a time**. All other processes must wait until the CPU becomes free and they are scheduled again.

The main objective of **multiprogramming** is to:

- Keep the CPU busy at all times
- Maximize CPU utilization

To achieve this:

- Several processes are kept in main memory
- When one process has to wait (usually for I/O), the OS:
    - Takes the CPU away from that process
    - Assigns the CPU to another ready process

This cycle continues, ensuring that the CPU rarely remains idle.

CPU Scheduling in Multicore Systems

On a **multicore system**, the same concept is extended:

- Each processing core should be kept busy
- Scheduling decisions are made independently for each core
- Improves overall system throughput and performance

CPU Scheduling – Need for Multiprogramming

In a simple system without multiprogramming:

- A process executes until it must wait (typically for I/O)
- During this waiting period, the CPU remains idle
- CPU time is wasted and no useful work is done

Multiprogramming allows the OS to:

- Switch the CPU to another ready process
- Use waiting time productively

This makes **CPU scheduling a fundamental OS function**.

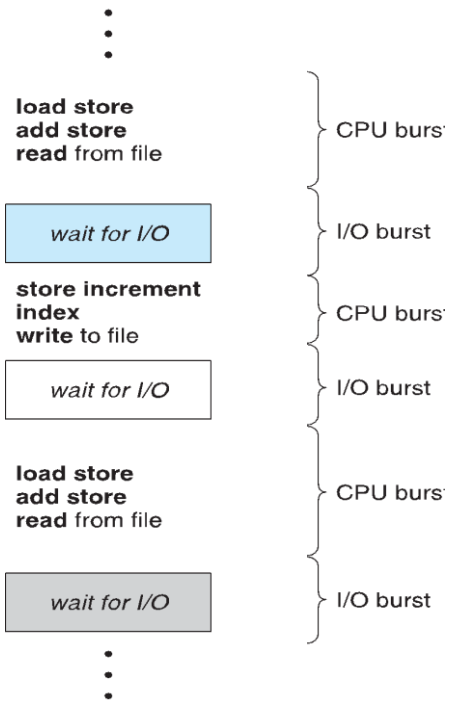
Alternating Sequence of CPU and I/O Bursts

Process execution consists of an **alternating sequence of CPU bursts and I/O bursts**.

- **CPU burst** → process is executing instructions
- **I/O burst** → process is waiting for I/O completion

Maximum CPU utilization is achieved when:

- CPU bursts of one process overlap with I/O bursts of another



CPU–I/O Burst Cycle

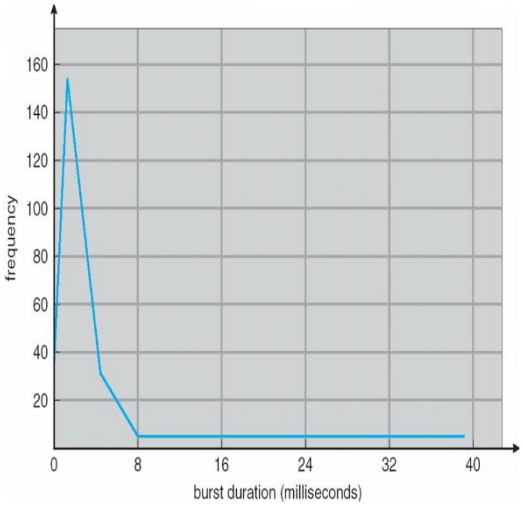
- Execution pattern:
  - CPU burst → I/O burst → CPU burst → I/O burst → …
- CPU scheduling algorithms focus mainly on:
  - The **distribution of CPU burst times**

Understanding this cycle is essential for designing efficient schedulers.

Histogram of CPU-Burst Times

The duration of CPU bursts:

- Varies widely among processes
- Follows a characteristic frequency distribution



Observations

- **I/O-bound processes**
  - Many short CPU bursts
- **CPU-bound processes**
  - Few long CPU bursts

This distribution is important when:

- Designing
- Evaluating
- Comparing CPU scheduling algorithms

CPU Scheduler

The **CPU scheduler** (short-term scheduler):

- Selects a process from the **ready queue**
- Allocates the CPU to that process

Ready Queue Organization

The ready queue may be organized in various ways:

- FIFO queue
- Priority queue
- Tree
- Unordered linked list

Each entry in the queue is a **Process Control Block (PCB)**.

Preemptive Scheduling

CPU scheduling decisions can occur when a process:

1. Switches from **running → waiting**
2. Switches from **running → ready**
3. Switches from **waiting → ready**
4. **Terminates**

Scheduling Types

- Cases **1 and 4 → Non-preemptive**
- Cases **2 and 3 → Preemptive**

Issues in Preemptive Scheduling

When preemption occurs:

- Shared data access must be handled carefully
- Preemption while executing in kernel mode must be considered
- Interrupts may occur during critical OS operations

Examples

- Windows 3.x → Non-preemptive
- Windows 95 onwards → Preemptive
- Macintosh OS → Uses preemptive scheduling

Preemptive vs Non-Preemptive Scheduling

Problems with Preemptive Scheduling

- Can cause **race conditions**

- Occurs when:
    - One process is preempted while updating shared data
    - Another process accesses inconsistent data

Solution

- Preemptive kernels use:
    - **Mutex locks**
    - Other synchronization mechanisms

Most modern operating systems are:

- **Fully preemptive**, even in kernel mode

Dispatcher

The **dispatcher** is a module that:

- Gives control of the CPU to the process selected by the short-term scheduler

Dispatcher Functions

- Performs **context switch**
- Switches CPU to **user mode**
- Jumps to the correct location in the user program

Dispatch Latency

- Time taken to:
    - Stop one process
    - Start another process
- Important performance metric

Scheduling Criteria

Scheduling algorithms are evaluated based on the following criteria:

1. **CPU Utilization**
    - Percentage of time CPU is busy
2. **Throughput**
    - Number of processes completed per unit time
3. **Turnaround Time**
    - Time from submission to completion of a process
4. **Waiting Time**
    - Total time a process spends waiting in the ready queue
5. **Response Time**

- Time from request submission to first response
- Important in time-sharing systems

Scheduling Algorithm Optimization Criteria

An ideal scheduling algorithm should aim to:

- **Maximize**
  - CPU utilization
  - Throughput
- **Minimize**
  - Turnaround time
  - Waiting time
  - Response time

Different scheduling algorithms optimize different criteria based on system goals.